

Common Data Types

 <https://github.com/learn-co-curriculum/python-p3-common-python-data-types>  <https://github.com/learn-co-curriculum/python-p3-common-python-data-types/issues/new>

Learning Goals

- Learn common data types in Python by comparing to similar and equivalent data types in JavaScript: `str` , `int` , `float` , `bool` , `list` , `tuple` , `set` , `dict` , `None` .

Key Vocab

- **Interpreter:** a program that executes other programs. Python programs require the Python interpreter to be installed on your computer so that they can be run.
- **Python Shell:** an interactive interpreter that can be accessed from the command line.
- **Data Type:** a specific kind of data. The Python interpreter uses these types to determine which actions can be performed on different data items.
- **Exception:** a type of error that can be predicted and handled without causing a program to crash.
- **Code Block:** a collection of code that is interpreted together. Python groups code blocks by indentation level.
- **Function:** a named code block that performs a sequence of actions when it is called.
- **Scope:** the area in your program where a specific variable can be called.

Introduction

Just like in JavaScript, Python has several common built-in data types for representing different kinds of information in our applications. In this lesson, we'll explore these different data types and see the similarities and differences in how Python and JavaScript treat these data types.

Make sure to follow along with the examples in this lesson in the Python shell! As an object-oriented language, Python gives you a lot of tools to inspect different data types, so you'll learn more by getting your hands on the code. Open the Python shell in your terminal by entering `python` or `python3`.

Strings

Like JavaScript, Python lets you define strings with either single quotes or double quotes:

```
"I'm a string"  
'Me too!'
```

You can also create a new string by using the built-in `str()` constructor function (though it's not common you'd need to):

```
str("I'm a string")
```

If you want to use string interpolation in Python, use an f-String like so:

```
# Python  
dog_name = "Lucy"  
print(f"Say hello to my dog {dog_name}")  
# => Say hello to my dog Lucy
```

This would be the equivalent of the following JavaScript code:

```
// JavaScript  
const dogName = "Lucy";  
console.log(`Say hello to my dog ${dogName}`);
```

NOTE: Backticks in Python are not valid characters, so **don't use backticks (`) for strings in Python.**

You can also use f-Strings to do more advanced formatting. Say you want to display all prices with two decimal places:

```
price_1 = 3
price_2 = 2.5

print(f"Item 1 costs ${price_1:.2f}")
# => Item 1 costs $3.00
print(f"Item 2 costs ${price_2:.2f}")
# => Item 2 costs $2.50
```

To see some more things you can do with strings in Python, open up the Python shell and run the following:

```
"hello"
# "hello"
"hello".upper()
# "HELLO"
"HELLO".lower()
# "hello"
"hello".capitalize()
# "Hello"
"hello" + "world"
# "helloworld"
"hello" * 3
# "hellohellohello"
```

You'll often hear it said that **"in Python, everything is an object"**. All of the methods that we called on strings above are available because the string literal "hello" is an **instance** of the `String` class. Thanks to Python's `type()` [function](https://www.geeksforgeeks.org/python-type-function/), you can see for yourself:

```
type("hello")
# => <class 'str'>
```

Using the `dir()` function on any Python object will display a list of all the methods that object responds to (you'll see `upper`, `lower`, `capitalize` and more in that list):

```
dir("hello")
# => ['__add__', '__class__', '__contains__', '__delattr__', '__dir__', ... ]
```

NOTE: Methods that are preceded and followed by double underscores are called **magic methods** or **dunder methods** (double **underscore**). Magic methods run automatically under certain conditions- we'll learn more about them when we cover object-oriented programming.

You can learn more about the many String methods by reading the [Python documentation](https://docs.python.org/3/library/string.html)  on Strings.

Numbers

In Python, unlike JavaScript, there are two types of numbers: Integers and Floats.

Integers are whole numbers, like `7`.

Floats are decimal numbers, like `7.3`.

There are a number of methods available to you for operating on or manipulating integers. You can read more about [Integers here](https://docs.python.org/3/library/functions.html#int)  [and more about Floats here](https://docs.python.org/3/library/functions.html#int)  [.](https://docs.python.org/3/library/functions.html#float)

You can convert some other data types to integers or floats with the `int()` and `float()` class constructor functions:

```
int("1")
# => 1
int(1.1)
# => 1
float("1.1")
# => 1.1
```

Like JavaScript, Python will convert an Integer to a Float when performing math operations. As you can see, the following calculations are equivalent:

```
4 / 3
# 1.3333333333333333
4 / 3.0
# 1.3333333333333333
float(4 / 3)
# 1.3333333333333333
```

Sequence Types

Python has a number of different data types that are useful for storing data. Each of these types can store any type of data inside; what differs between them are the rules for organizing and accessing the data.

Lists

There are a number of ways to create a list. Just like with creating strings, you can use the literal constructor or the class constructor.

`[1, 3, 400, 7]` is a list of integers. Any set of comma separated data enclosed in brackets is a list. So, by simply writing something like the above, you can create a list:

```
[1, 3, 400, 7]
# => [1, 3, 400, 7]
```

You can also create an list with the `list()` [syntax](https://docs.python.org/3/library/stdtypes.html#list)  `(https://docs.python.org/3/library/stdtypes.html#list)`. Just typing `list()` will create an empty list (`[]`):

```
list()
# => []
```

In order to access a specific element in a list, you will need to know its **index**, or the place it occupies in the list. **Indices** start at 0 and move up by 1 with each subsequent element. Once the index is known, the element can be accessed using square brackets and the index.

```
list_abc = ['a', 'b', 'c']  
list_abc[0]  
# => 'a'  
list_abc[1]  
# => 'b'  
list_abc[2]  
# => 'c'
```

There are many ways to operate on lists and on each individual item, or element, within an list. Later on in the course, we'll learn about methods for iterating over lists (as with the `.forEach`, `.map`, etc methods in JavaScript). For now, we'll preview a few list functions and methods, and you can check out more [here](https://docs.python.org/3/library/stdtypes.html#list)  (<https://docs.python.org/3/library/stdtypes.html#list>).

```
len([1, 3, 400, 7])  
# 4  
sorted([5, 100, 234, 7, 2])  
# [2, 5, 7, 100, 234]  
list_123 = [1, 2, 3]  
list_123.pop()  
# 3  
list_123.remove(1)  
print(list_123)  
# [2]
```

Tuples

Tuples are nearly identical to lists, with two key differences:

First, tuples are created with open and close parentheses (in place of square brackets). The `tuple()` class constructor function can also be used to cast lists and other iterable data types to tuples.

```
(1, 2, 3)
# => (1, 2, 3)
tuple([1, 2, 3])
# => (1, 2, 3)
```

Second, tuples are **immutable**. This means that once a tuple has been created, the tuple itself cannot be changed. Python functions that work on lists to create new data will still work on tuples, but tuples do not contain methods like `.pop()` and `.insert()` that would change their contents.

While tuples are less flexible than lists, this can prove advantageous in certain situations. The most common situation where you will see tuples while at Flatiron will be in data retrieved from a database. Since you want to keep an accurate record of what is in the database while your application works with the data, a tuple will protect that information until it is no longer needed.

NOTE: Parentheses can also be used for order of operations and grouping statements. To let Python know that it's looking at a tuple, there has to be at least one comma- even in tuples with only one element: `(1,)`.

For more on sequence data types, check [python.org's sequence documentation](https://docs.python.org/3/library/stdtypes.html#sequence-types-list-tuple-range)  (<https://docs.python.org/3/library/stdtypes.html#sequence-types-list-tuple-range>)

Sets and Dicts

Sets and dicts in Python also store sequences of data, but the individual elements in sets and dicts are **unique**.

Sets

A [set](https://docs.python.org/3/library/stdtypes.html#set)  (<https://docs.python.org/3/library/stdtypes.html#set>) is unindexed, unordered, and unchangeable. It can be initiated with curly brackets or the `set()` class constructor. The `set()` class constructor takes a list or tuple as its only argument (remember those brackets and parentheses!) The elements of a set are unique:

```
set()
# => {}

set(3, 2, 3, 'a', 'b', 'a')
# => TypeError: set expected at most 1 argument, got 6

set([3, 2, 3, 'a', 'b', 'a'])
# => {2, 3, 'a', 'b'}
```

Unindexed means that we cannot access elements of the set using square brackets as we do in lists and tuples.

Unordered means that the contents of the set are in a random order.

Unchangeable means that the individual elements of a set cannot be changed.

NOTE: *Immutable* and *unchangeable* mean different things when we're talking about data types in Python. A set is not immutable because its overall structure can be changed; it can be made shorter or longer. It is unchangeable because an element cannot be changed into something else.

Sets have many of the same methods as lists:

```
s = {1, 2, 3}
s.pop()
# => 1
s.remove(3)
# => {2}
```

Dictionaries

Dictionaries are Python's equivalent of a plain old JavaScript object. They are composed of key/value pairs. Each key points to a specific value, just like a word and a definition in a regular dictionary.

You can create a dictionary by simply writing key/value pairs enclosed in curly brackets. Note that keys must be in string format:


```
{ "key1": "value1", "key2": "value2" }
```

To access data from this dictionary, you can use the square bracket notation and pass in the symbol for the key you are trying to access:

```
my_dict = { "key1": 1, "key2": 2 }  
my_dict["key2"]  
# => 2
```

You can also use the built-in `.get()` [method](https://www.w3schools.com/python/ref_dictionary_get.asp) to retrieve the value for a key. This is a useful method for times when you're unsure if a key exists, as it returns `None` instead of an error if no matching key exists:

```
my_dict = { "key1": "value1", "key2": "value2" }  
print(my_dict["key3"])  
# => KeyError: 'key3'  
  
print(my_dict.get("key3"))  
# => None
```

Unlike JavaScript, you cannot use the dot notation to access keys on dictionaries — only the bracket notation will work:

```
my_dict = { "key1": "value1", "key2": "value2" }  
my_dict.key2  
# => AttributeError: 'dict' object has no attribute 'key2'
```

You can also create dictionaries using the `dict()` [class constructor](https://docs.python.org/3/tutorial/datastructures.html#dictionaries).

```
dict(x = 1, y = 2)  
# => {'x': 1, 'y': 2}
```

There are many methods for operating on dictionaries and their individual key/value pairs. We will learn much more about them later, but you can preview some methods [here](https://docs.python.org/3/tutorial/datastructures.html#dictionaries).

None

In Python, there is one special value that represents the **absence** of a value, `None` .

In JavaScript, there are two different data types for representing the absence of value: `null` and `undefined` :

```
let noValue;  
console.log(noValue);  
// => undefined  
noValue = null;  
console.log(noValue);  
// => null
```

`undefined` in JavaScript generally comes up in a few places: when a variable has been created, but hasn't been assigned a value, and when a function doesn't have any return value. `null` , on the other hand, is used to explicitly signify the absence of any value.

Unlike JavaScript, Python won't let you create a variable without assigning a value. You must explicitly assign a value of `None` if you want an "empty" variable:

```
no_value  
# => NameError: name 'no_value' is not defined  
  
no_value = None  
print(no_value)  
# => None
```

Booleans

There are only two values of the Boolean data type: `True` and `False` . We can confirm this by inspecting `True` and `False` with the `type()` function.

```
type(True)
# => <class 'bool'>
type(False)
# => <class 'bool'>
```

Python, like JavaScript, has the concept of "truthy" and "falsy" values as well: values which, when coerced to their equivalent boolean value, or evaluated as part of a conditional statement, return either `True` or `False` :

```
not True
# => False
not False
# => True
not 1
# => False
not 0
# => True
not ''
# => True
not None
# => True
not []
# => True
not ()
# => True
not {}
# => True
```

NOTE: `not` is the operator that reverses the truth value of a value, variable, or statement. `!` still plays a role in Python, but it is only used in the `!=` operator that asserts that two values are not equal.

Like JavaScript, Python has many falsy values. They do not map perfectly to one another, though.

Let's look back at JavaScript, where `null` , `undefined` , `false` , `0` , `NaN` , and `""` are all falsy values:

```
!!null;  
// => false  
!!undefined;  
// => false  
!!false;  
// => false  
!!0;  
// => false  
!!NaN;  
// => false  
!!"";  
// => false
```

Conclusion

One of the first things to familiarize yourself with when learning a new language is its common data types. You'll find similarities across almost all programming languages when it comes to data types, with some differences of opinion cropping up as well, like what data is considered "truthy" and "falsy".

As you're exploring data types in Python, make sure to keep the "everything is an object" principle in mind, and take advantage of methods that let you ask questions about your Python data like its attributes and methods. Keep the [Python documentation](https://docs.python.org/3/)  [\(https://docs.python.org/3/\)](https://docs.python.org/3/) handy too!

Resources

- [Switching to Python](https://realpython.com/switching-to-python/)  [\(https://realpython.com/switching-to-python/\)](https://realpython.com/switching-to-python/)
- [Python Data Types](https://www.w3schools.com/python/python_datatypes.asp)  [\(https://www.w3schools.com/python/python_datatypes.asp\)](https://www.w3schools.com/python/python_datatypes.asp)

- [Python documentation](https://docs.python.org/3/)  [\(https://docs.python.org/3/\)](https://docs.python.org/3/)